A E R O S T A C K 2

Dynamic Multi-Drone Area Survey

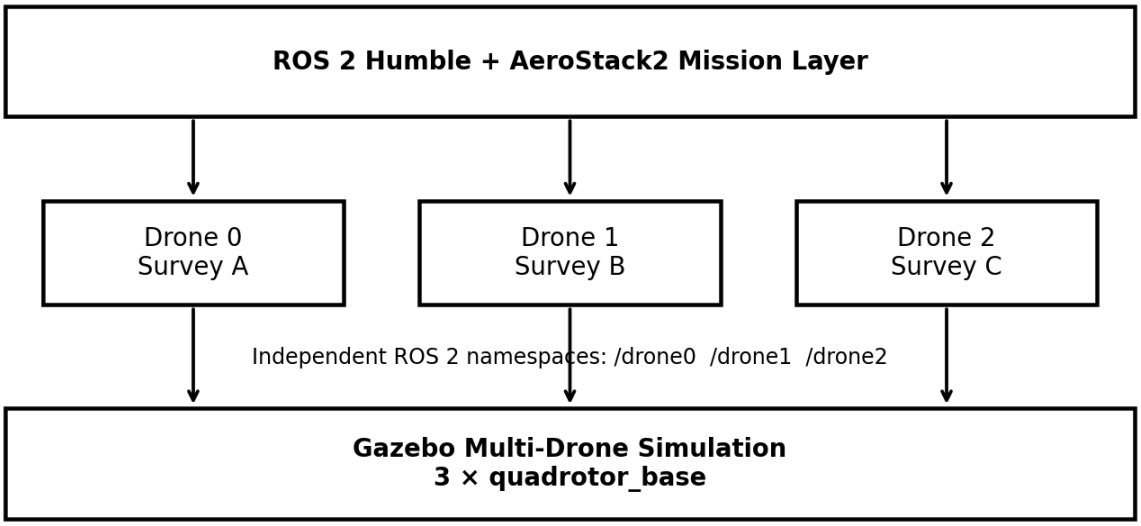


Figure 1. High-level architecture of the three-drone AeroStack2 survey system.

2. Executive Summary

This project implements a three-drone autonomous area-survey mission using AeroStack2, ROS 2 Humble and Gazebo. The survey field is divided into three independent regions, initially assigned to drone0, drone1 and drone2. The three drones take off and begin surveying their assigned regions concurrently. The central mission manager monitors progress and is able to detect a deliberately simulated delay on drone0, identify its unfinished waypoints, select a helper, preempt a busy helper when no idle drone is available, transfer the unfinished work, and then resume the helper's original survey mission.

The final validated run demonstrated the complete recovery sequence. Drone0 completed two of four Area A waypoints and then entered an intentionally simulated delay before waypoint A3. The manager identified A3 and A4 as unfinished. Because no helper became naturally available within the configured waiting period, drone2 was selected and its active survey was preempted. Drone2 then completed A3 and A4 on behalf of drone0 and subsequently resumed its original Area C survey.

The mission reached full coverage verification with Area A = 4/4, Area B = 4/4 and Area C = 4/4. The system reported dynamic reallocation as completed, identified drone2 as the helper, successfully landed all three drones and terminated with “MISSION COMPLETED SUCCESSFULLY.”

The successful console run is the primary validation evidence for this report. The implementation uses AeroStack2 DroneInterface objects under the /drone0, /drone1 and /drone2 namespaces and uses asynchronous GoTo execution for task-level preemption and reallocation.

Key Demonstrated Capabilities

- Three independent AeroStack2 DroneInterface instances operating in one ROS 2 process.
- Concurrent takeoff and parallel survey execution.
- Geographic task decomposition into three survey regions.
- Waypoint-level progress tracking and completion accounting.
- Detection of a simulated drone delay during mission execution.
- Identification of unfinished work and dynamic task transfer.
- Preemption of an actively surveying helper when no idle drone is available.
- Completion of transferred work followed by resumption of the helper's original assignment.
- Independent final verification of all three survey regions before landing.

Mission Event Timeline

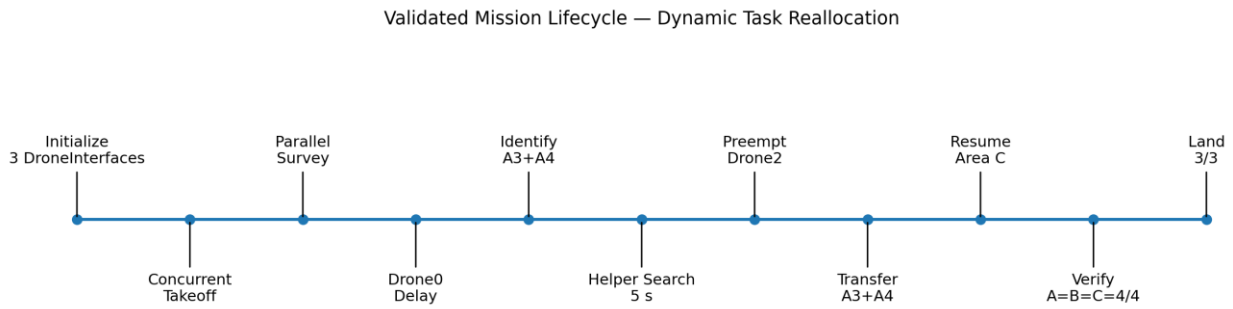


Figure. End-to-end mission lifecycle captured by the validated implementation.

3. System Overview

The system is built entirely around AeroStack2 running over ROS 2 Humble, with Gazebo providing the simulated multi-quadrotor environment. Each simulated vehicle is exposed through its own ROS 2 namespace and controlled through an AeroStack2 DroneInterface.

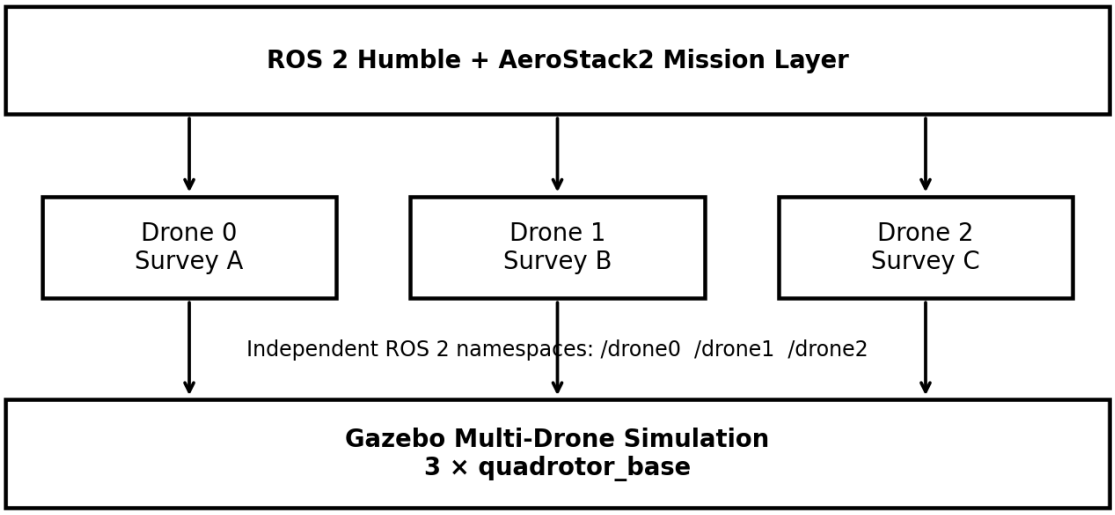


Figure 2. Mission-control architecture: one coordinating Python process controls three independent drone namespaces.

The mission manager sits above the DroneInterface layer. Its responsibility is not to fly the drones at the low level; instead, it manages mission state, waypoint ownership, completion accounting, delay handling, helper selection, preemption, work transfer and final mission verification.

Layer	Responsibility
Mission Manager	Coordinates assignments, detects delay, reallocates unfinished work and verifies coverage.
Survey Drone object	Tracks state, waypoint progress, original assignment and preemption state.
AeroStack2 DroneInterface	Provides arm, offboard, takeoff, GoTo and land behaviors.
ROS 2	Provides namespaces, actions, topics and communication between mission code and vehicle stack.
Gazebo	Provides the physics-based simulated three-drone environment.

4. Software and Simulation Environment

Item	Configuration
Operating System	Ubuntu 22.04.5 LTS
Middleware	ROS 2 Humble
Simulator	Gazebo
Framework	AeroStack2
Programming	Python 3 / rclpy
Drone API	AeroStack2 DroneInterface
AeroStack2 workspace	~/aerostack2_ws
Project root	~/project_gazebo
Main mission script	area_survey.py
World configuration	config/world_swarm.yaml
Drone model	quadrotor_base

The environment setup follows the same reproducible sourcing sequence used in the reference AeroStack2 project document: ROS 2 Humble is sourced first, followed by the AeroStack2 workspace overlay, and then the project directory is entered. The reference document identifies ~/aerostack2_ws as the correct AeroStack2 overlay and ~/project_gazebo as the project root.

```
source /opt/ros/humble/setup.bash
source ~/aerostack2_ws/install/setup.bash
cd ~/project_gazebo
```

ROS 2 Multi-Agent Namespaces

Namespace	Role in final mission	Initial area
/drone0	Primary survey drone / delayed agent	Area A
/drone1	Independent survey drone	Area B
/drone2	Survey drone / reallocation helper	Area C

5. Three-Drone Survey Configuration

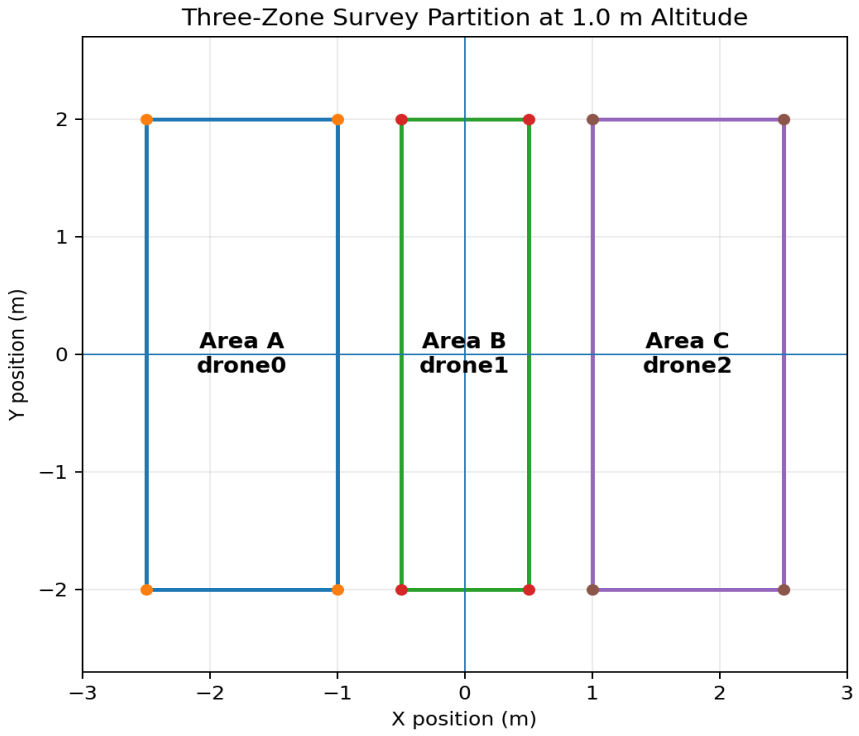


Figure 3. Survey-field partition used by the final validated mission.

Drone	Assignment	Survey waypoints (x, y, z) m

drone0	Area A	(-2.5,-2.0,1.0) → (-2.5,2.0,1.0) → (-1.0,2.0,1.0) → (-1.0,-2.0,1.0)
drone1	Area B	(-0.5,-2.0,1.0) → (-0.5,2.0,1.0) → (0.5,2.0,1.0) → (0.5,-2.0,1.0)
drone2	Area C	(1.0,-2.0,1.0) → (1.0,2.0,1.0) → (2.5,2.0,1.0) → (2.5,-2.0,1.0)

The final run used four waypoints per region at a common altitude of 1.0 m. The three regions are arranged as adjacent strips so that the complete field is covered without requiring formation flight.

Survey Assignment Model

Each waypoint belongs to an area-level work package. A waypoint is considered complete only after its GoTo behavior finishes successfully. This makes unfinished work explicit and transferable rather than treating a whole region as an indivisible task.

The initial allocation is deliberately simple: drone0 → A, drone1 → B and drone2 → C. Dynamic reallocation changes only the ownership of unfinished work; completed waypoints remain completed and are never reissued.

6. Mission Parameters

Parameter	Value	Purpose
Takeoff height	1.0 m	Common survey altitude.
Takeoff speed	0.7 m/s	Initial ascent command.
Mission speed	1.0 m/s	GoTo speed used by survey segments.
Landing speed	0.4 m/s	Descent command.
Stabilization time	3.0 s	Initial airborne stabilization before survey.
Simulated delay	8.0 s	Intentional delay used to exercise recovery.
Delay drone	drone0	Agent used to trigger the reallocation event.
Delay point	Before waypoint 3	Leaves A3 and A4 unfinished.
Helper wait timeout	5.0 s	Time allowed for a naturally available helper.
Preemption timeout	8.0 s	Configured stop/recovery window.
GoTo execution	Asynchronous	Required for responsive preemption and task transfer.

The delay is a controlled simulation condition rather than a claim of autonomous hardware fault diagnosis. It is used to validate the mission manager's recovery behavior in a repeatable environment.

Mission State Model

State	Meaning
SURVEYING	Drone is executing its currently assigned waypoint sequence.
DELAYED	Drone has reached the configured simulated delay condition.
AVAILABLE	Drone has completed its current original work and can accept additional work.
PREEMPTED	Drone's current survey behavior has been intentionally stopped.
REASSIGNED	Drone is executing transferred work belonging to another drone.
TRANSFERRED	Original delayed drone's unfinished work has been handed to a helper.

7. Dynamic Task Reallocation Strategy

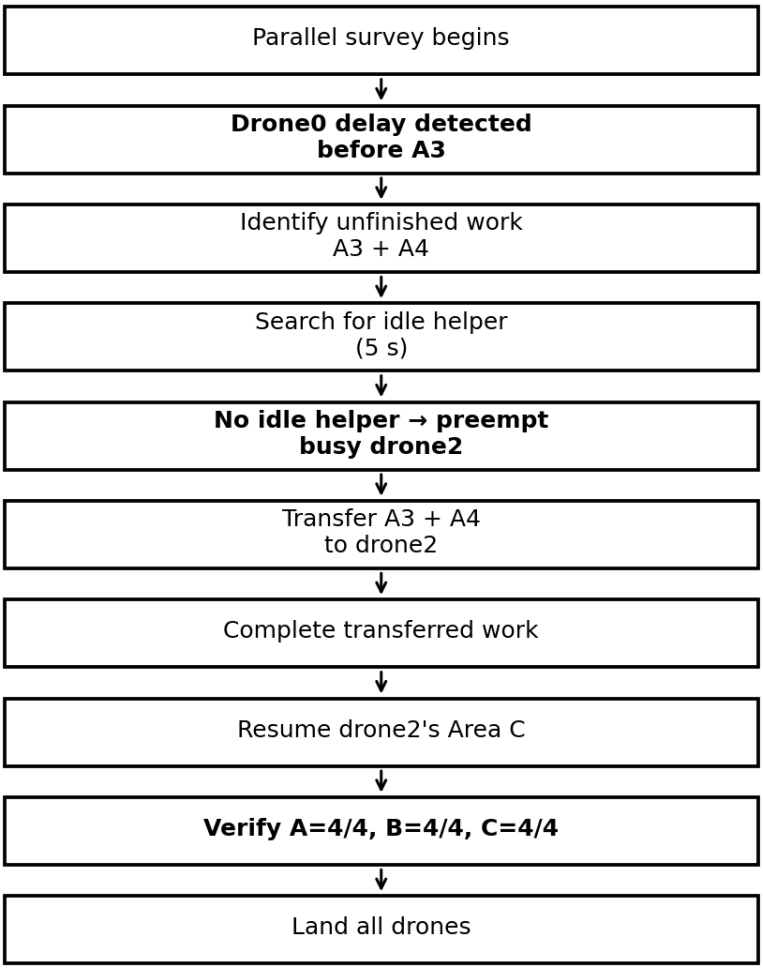


Figure 4. Dynamic recovery workflow implemented by the mission manager.

8. Implementation Architecture

Component	Responsibility
SurveyDrone (DroneInterface subclass)	Represents one simulated vehicle and stores mission-specific state.
execute_go_to()	Starts an asynchronous GoTo operation and waits by monitoring the behavior state.
run_initial_survey()	Executes the drone's original area waypoint sequence.
stop_current_behavior()	Requests preemption of the active GoTo behavior.
get_remaining_indices()	Returns original waypoints not yet completed.
execute_reassigned_task()	Executes transferred waypoints on the helper drone.
MissionManager	Coordinates all drones, delay detection, helper selection and final verification.
perform_reallocation()	Transfers delayed work to an available or preempted helper.
mark_complete()	Maintains global area-level completion accounting.
land_all()	Commands concurrent landing after full survey verification.

8.1 Asynchronous GoTo Execution

A key implementation detail is the use of the AeroStack2 GoTo module in asynchronous mode. A blocking GoTo call would prevent the mission manager from reacting cleanly to preemption and task transfer. The final implementation starts the GoTo behavior without waiting for the action result, then monitors the behavior until it becomes idle or the mission requests preemption.

```
accepted = self.go_to(
    x, y, z,
    MISSION_SPEED,
    0, None,
    "earth",
    False          # wait=False
)

while self.go_to.is_running():
    time.sleep(0.1)
```

8.2 Preemption Semantics

Preemption is applied at the active waypoint level. The helper's current waypoint is not marked complete when its GoTo is interrupted. After the behavior is stopped, the helper's completed waypoint set is preserved and the remaining original indices are reconstructed. This prevents duplicate execution and makes the transferred workload deterministic.

8.3 Global Completion Accounting

The manager maintains completion separately from drone ownership. This distinction is important: Area A can be completed by more than one physical drone over the course of a mission, but each Area A waypoint should count only once. The final verification therefore checks area-level completion rather than simply counting how many waypoints each drone personally executed.

9. Project Structure

```
~/project_gazebo/
├── launch_as2.bash
├── launch_ground_station.bash
├── stop.bash
├── area_survey.py
├── area_survey_WORKING_FINAL_25Sep2026.py
├── mission_swarm.py
└── config/
    └── world_swarm.yaml
```

File	Purpose
launch_as2.bash	Starts AeroStack2 + Gazebo in multi-drone mode.
launch_ground_station.bash	Optional live ground-station monitoring.
stop.bash	Stops the simulation and associated processes.
area_survey.py	Main dynamic multi-drone area-survey mission.
config/world_swarm.yaml	Defines the three-drone Gazebo world configuration.
mission_swarm.py	Existing AeroStack2 swarm/mission utility retained in the project workspace.

The reference AeroStack2 document uses the same launch, ground-station and stop scripts and identifies the project root, world configuration and mission-script structure.

9.1 Environment Setup

```
source /opt/ros/humble/setup.bash
source ~/aerostack2_ws/install/setup.bash
cd ~/project_gazebo
```

Only the AeroStack2 workspace overlay is sourced; the project directory itself is not treated as a ROS 2 overlay. This matches the validated environment procedure in the reference project document.

10. Simulation Launch and Verification

10.1 Launch Multi-Drone Simulation

```
./launch_as2.bash -m
```

This launches the Gazebo world and the AeroStack2 stack for the three configured drone namespaces. The reference project document describes -m as the multi-drone launch mode.

10.2 Ground Station

```
./launch_ground_station.bash -m -v
```

10.3 Run the Dynamic Mission

```
python3 area_survey.py
```

10.4 Stop the Simulation

```
./stop.bash
```

10.5 ROS 2 Interface Verification

```
for d in drone0 drone1 drone2; do
    echo "========== $d =========="
    ros2 topic echo /$d/platform/info --once
done
```

The same /platform/info verification method is used in the reference document to confirm that each simulated drone is connected before mission commands are issued.

Check	Expected
drone0 interface	Initialized
drone1 interface	Initialized
drone2 interface	Initialized
GoToBehavior	Available for each drone
TakeoffBehavior	Available for each drone
Gazebo entities	drone0, drone1, drone2 present

11. Mission Execution Procedure

Operation
Source ROS 2 and AeroStack2 overlays.
Enter ~/project_gazebo.
Launch Gazebo/AeroStack2 in multi-drone mode.
Initialize drone0, drone1 and drone2 interfaces.
Arm all drones and enable Offboard control.
Take off concurrently to 1.0 m.
Survey Areas A, B and C concurrently.
Trigger the controlled 8-second delay on drone0 before A3.
Identify A3/A4 as unfinished.
Wait 5 seconds for a naturally available helper.
If none is available, preempt drone2.
Transfer A3/A4 to drone2.
Complete transferred Area A work.
Resume drone2's original Area C work.
Verify A=4/4, B=4/4, C=4/4.
Land all drones concurrently and shut down interfaces.

The final run followed this lifecycle successfully. The initial three-drone takeoff and parallel survey are visible in the console output, followed by the mission manager activation and subsequent recovery sequence.

12. Dynamic Reallocation Event — Validated Run

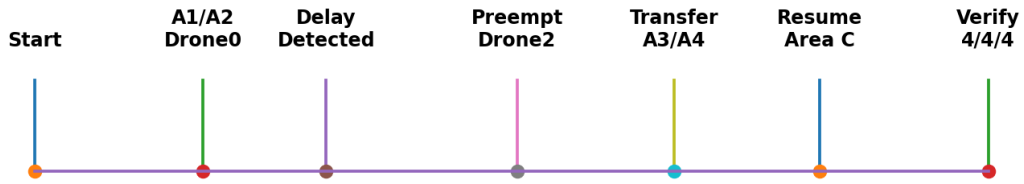


Figure 5. Major events recorded during the successful dynamic-reallocation run.

Stage	Observed result
Initial survey	drone0→A, drone1→B, drone2→C; all three started concurrently.
Drone0 progress	A1 and A2 completed; Area A reached 2/4.
Delay event	8-second simulated delay before A3.
Unfinished work	A3 and A4 identified.
Helper search	No idle helper became available within 5 seconds.
Preemption	drone2 selected and its active survey GoTo stopped.
Transfer	A3 and A4 transferred from drone0 to drone2.
Transferred execution	drone2 completed A3 and A4.
Original-work recovery	drone2 resumed Area C and completed its remaining waypoints.
Verification	A=4/4, B=4/4, C=4/4.
Landing	drone0, drone1 and drone2 all reported LAND SUCCESS.

The run therefore demonstrates both forms of recovery: the delayed drone's unfinished work is transferred to another agent, while the helper's own interrupted workload is retained and resumed afterward. This is more than simple task reassignment because work ownership is temporarily changed without losing the helper's original mission state.

13. Validation and Results

Validation category	Result
Drone interface initialization	drone0, drone1 and drone2 initialized successfully.
Concurrent takeoff	All three drones reported TAKEOFF SUCCESS.
Parallel survey start	All three assigned regions began execution.
Delay detection	Drone0 delay detected before waypoint A3.
Unfinished-work identification	A3 and A4 correctly identified.
Natural helper search	No helper available within configured 5 s window.
Preemptive helper selection	drone2 selected.
Active behavior stop	drone2's current GoTo was successfully stopped.
Task transfer	A3 and A4 transferred to drone2.
Transferred work	A3 and A4 completed by drone2.
Original work resumption	drone2 resumed Area C.
Area A verification	4/4.
Area B verification	4/4.
Area C verification	4/4.
Dynamic reallocation	COMPLETED.
Landing	All three drones reported LAND SUCCESS.
Mission termination	MISSION COMPLETED SUCCESSFULLY.

The final console output explicitly reports Area A = 4/4, Area B = 4/4 and Area C = 4/4, identifies drone2 as the helper, declares dynamic reallocation completed, and reports successful landing of all three drones.

14. Core Working Source — Implementation Excerpts

The executable used for the successful run is area_survey.py. The report records the critical implementation patterns below; the project file remains the authoritative source for the complete implementation.

14.1 Drone Interface

```
class SurveyDrone(DroneInterface):
    def __init__(self, namespace):
        super().__init__(
            namespace,
            verbose=True,
            use_sim_time=True
        )
        self.mission_state = "IDLE"
        self.completed_indices = set()
        self.preempt_requested = False
```

14.2 Survey Waypoint Definitions

```
AREA_A = [
    (-2.5, -2.0, 1.0),
    (-2.5,  2.0, 1.0),
    (-1.0,  2.0, 1.0),
    (-1.0, -2.0, 1.0),
]

AREA_B = [
    (-0.5, -2.0, 1.0),
    (-0.5,  2.0, 1.0),
    ( 0.5,  2.0, 1.0),
    ( 0.5, -2.0, 1.0),
]

AREA_C = [
    (1.0, -2.0, 1.0),
    (1.0,  2.0, 1.0),
    (2.5,  2.0, 1.0),
    (2.5, -2.0, 1.0),
]
```

14.3 Asynchronous GoTo

```
accepted = self.go_to(
    x, y, z,
    MISSION_SPEED,
    0,
    None,
    "earth",
    False
)

if not accepted:
    return False

while self.go_to.is_running():
    if self.preempt_requested:
        # manager will stop the active behavior
        pass
    time.sleep(0.1)
```

14.4 Reallocation Logic — Conceptual Core

```
remaining = delayed_drone.get_remaining_indices()

helper = find_available_helper()

if helper is None:
    helper = select_busy_helper()
    helper.preempt_requested = True
    helper.stop_current_behavior()

transfer(delayed_drone, helper, remaining)

execute_reassigned_task(helper, remaining)

resume_original_survey(helper)
```

These excerpts illustrate the architecture and algorithm; they are intentionally concise. The complete executable is retained in the project workspace and was the source of the successful 26 September 2026 validation run.

15. Reproducibility — Important Terminal Commands

15.1 Environment

```
source /opt/ros/humble/setup.bash
source ~/aerostack2_ws/install/setup.bash
cd ~/project_gazebo
```

15.2 Launch

```
./launch_as2.bash -m
```

15.3 Optional Monitoring

```
./launch_ground_station.bash -m -v
```

15.4 Verify Nodes

```
ros2 node list | grep -E '/drone0|/drone1|/drone2'
```

15.5 Verify Actions

```
ros2 action list | grep -E '/drone0|/drone1|/drone2'
```

15.6 Run Mission

```
python3 area_survey.py
```

15.7 Stop

```
./stop.bash
```

15.8 Syntax Check

```
python3 -m py_compile area_survey.py
```

The reference project document follows the same environment, launch and stop command sequence and documents the reproducibility of the simulation workflow.

16. Conclusion

The Dynamic Multi-Drone Area Survey and Task Reallocation project successfully demonstrates mission-level resilience for a three-drone AeroStack2 simulation. The system begins with independent area assignments, executes the survey concurrently, detects a controlled delay, identifies unfinished work, preempts a busy helper when required, transfers the unfinished work, and then resumes the helper's original mission.

The final validation is especially significant because the helper did not simply finish the transferred work and stop. Drone2 completed drone0's remaining Area A work and then resumed its own Area C workload. The mission manager subsequently verified full coverage across all three areas before issuing the landing commands.

The successful run therefore validates the complete intended lifecycle: interface initialization → arming → Offboard → concurrent takeoff → parallel area survey → delay detection → helper search → preemption → task transfer → transferred execution → original-task resumption → area verification → landing → clean shutdown.

17. Validation Evidence Summary

Evidence point	Recorded observation
Initialization	All three DroneInterface objects initialized.
Takeoff	drone0, drone1 and drone2 reported TAKEOFF SUCCESS.
Delay	drone0 delayed before waypoint 3 after completing waypoint 2.
Remaining work	A3 and A4 were explicitly listed.
Helper selection	drone2 selected after no idle helper became available.
Preemption	drone2 active GoTo was stopped successfully.
Transfer	Area A waypoints 3 and 4 transferred to drone2.
Completion	drone2 completed transferred Area A waypoints.
Resumption	drone2 resumed original Area C mission.
Final verification	Area A 4/4, Area B 4/4, Area C 4/4.
Landing	All three drones reported LAND SUCCESS.
Termination	MISSION COMPLETED SUCCESSFULLY.

The final output contains an explicit success statement after shutdown, making the run independently traceable from mission start to mission completion.

18. Final Validation & Visual Evidence

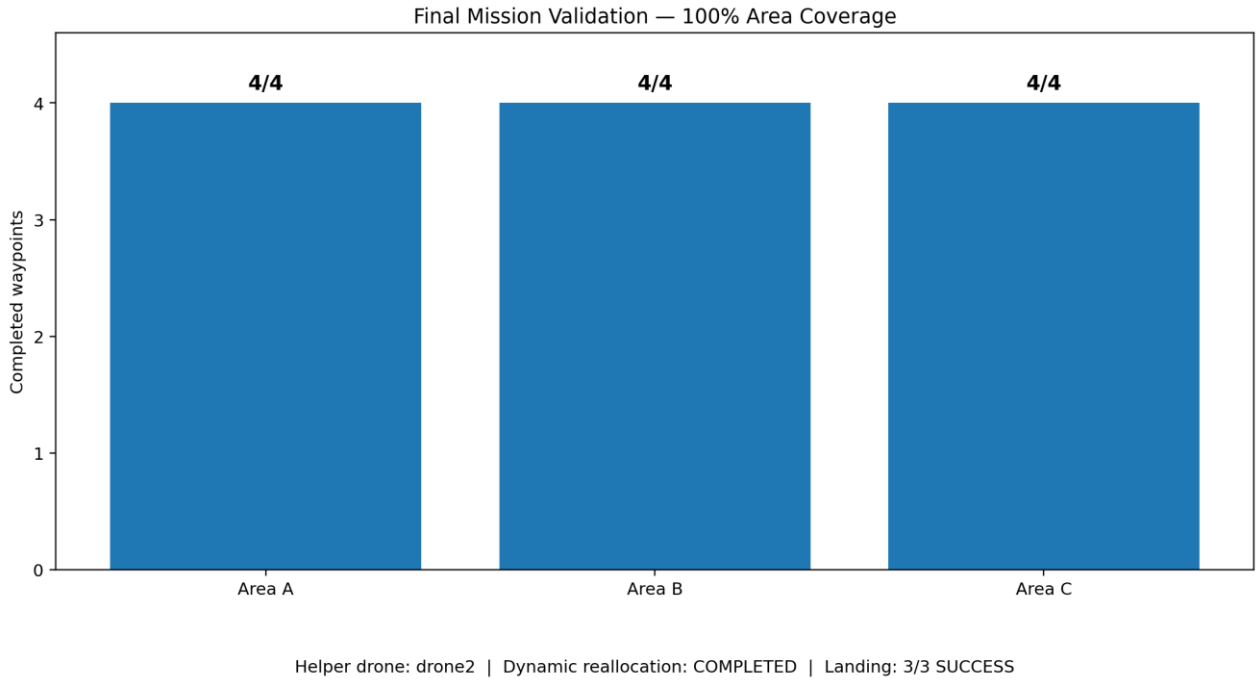


Figure 6. Final validation dashboard from the successful simulation run.

Validation Item	Observed Result	Status
Drone interfaces	drone0, drone1 and drone2 initialized	PASS
Takeoff	All three drones reported TAKEOFF SUCCESS	PASS
Parallel survey	A, B and C started concurrently	PASS
Delay event	Drone0 delayed before A3 after A1 and A2	PASS
Unfinished work	A3 and A4 identified	PASS
Helper search	No idle helper during 5 s window	EXPECTED
Preemption	Drone2 active GoTo stopped successfully	PASS
Transfer	A3 and A4 assigned to drone2	PASS
Transferred execution	Drone2 completed A3 and A4	PASS
Original-work recovery	Drone2 resumed and completed Area C	PASS
Final coverage	Area A 4/4, Area B 4/4, Area C 4/4	PASS
Landing	All three drones reported LAND SUCCESS	PASS
Termination	MISSION COMPLETED SUCCESSFULLY	PASS